



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

BIT 2114 Object Oriented Analysis and Design

LAST REVISION ON April 28, 2014





This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 7

Domain Modeling

Learning outcomes

Upon completing this topic, you should be able to:

1. Explain the importance of domain modeling.
2. Describe the different models used.
3. Model the requirements from the use case diagram to all of the models examined.



7.1. UML Domain Modeling

Domain modeling is used to model the use cases. It is an abstract representation of a specification design, aiming to express the essentials of a situation without giving unnecessary information. The UML language is designed to express the static as well as the dynamic aspects of a system. The static part is represented mainly by the class diagram and defined by the classes, attributes and the relationships among classes. Sequence and class diagrams are discussed below.

7.2. Class Diagram

Class diagrams allow us to express classes in UML, including attributes and methods. Class diagrams allow us to declare our classes (visually). It is a UML diagram that shows classes with



their attributes and operations together with associations between classes.

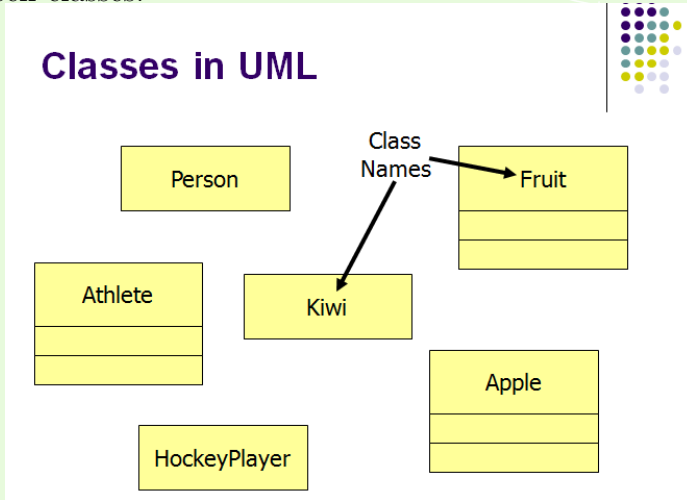


Figure 7.1: Classes in UML



BIT 2114 Object Oriented Analysis and Design

As you can imagine, the class shapes shown here are a major part of class diagrams:

1. Both forms are acceptable for classes that have no attributes or methods.
2. The two empty blocks are used for holding declarations of attributes and methods.
3. The top block is used to hold the class name.

7.2.1. Adding Attributes

To find attributes, the real-world attributes of objects can be used as well as the attributes necessary to fulfill the requirements defined by the use case under consideration.



Attributes in Classes

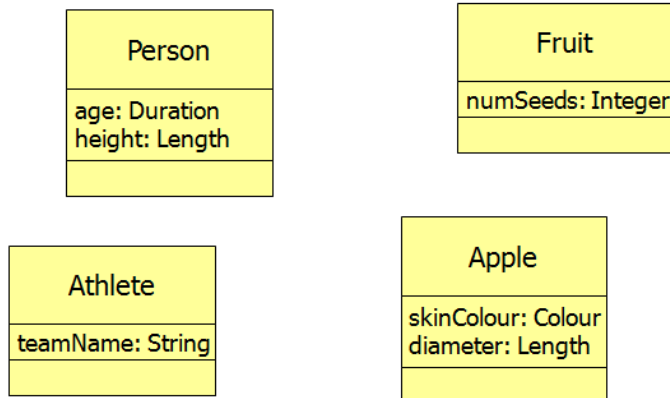


Figure 7.2: Attributes in Classes

Attributes are specified using the form:



- name: type.

This format is taken from Eiffel, which uses a Pascal-like syntax. Names are usually alphanumeric. Types are logical types from the domain. In this example, an apple's diameter is a quantity of type 'Length', which could be represented as a float or double in C++ or Java. Attributes can be read/write (default) or read only. Attributes that are read only are preceded with the modifier '/'. Attributes can be defined on the objects (default) or the class. Class attributes are preceded with the modifier '\$'.

Attributes can have different visibilities: Public attributes, denoted with the modifier '+', can be accessed from outside the class. Private attributes, denoted with the modifier '-', can only be accessed from within the class. Protected attributes, denoted with the modifier '#', can be accessed from within the class, or



any descendant (e.g. a subclass).

7.2.2. Operations

Operations can be defined, using the form:

- `opName([in|out|inout] param1 : type1,...) : returnType`

The modifiers in, out, and inout specify which direction(s) the parameters are to travel

- in: input to the operation
- out: output from the operation
- inout: input to the operation, possibly modified, then output from the operation

Often, if no modifier is specified, a parameter is assumed to be input only (in). Again, `type1` and `returnType` are types from



the application domain. Again, type1 and returnType are types from the application domain.

Abstract Classes and Methods

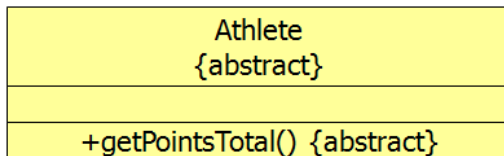


Figure 7.3: Abstract Class

To identify classes and create a class diagram:

1. list the candidate classes using the noun phrase identification related to the current requirements under considera-



- tion,
2. list the identified classes and draw them into the class diagram,
 3. add the associations necessary to record relationships among classes,
 4. add the attributes to fulfill the information requirements

7.3. Class Diagram Relationships

To satisfy the information requirement of the use cases under development, it is necessary to identify the associations between classes. A relationship is a semantic connection between objects of one or more classes. They are generally labeled with verb phrases. A relationship generally is described by its name,



degree and multiplicity.

- Degree of a relationship
 - Number of classes that participate in that relationship
- The cardinality of object A is the number of objects of class A that can (or must) be associated with each object of class B
- Multiplicity in relationships (Multiplicity of object A)
 - The range of the number of objects of class A that can (or must) be associated with each object of class B
- There are three types of relationships:
 - A 'is a type of' B (inheritance)



- A ‘is associated with’ B (association)
- A ‘is a part of’ B (composition)

The technique of an association list is helpful for identifying relationships.

7.3.1. Class Diagram: Generalization

A superclass/subclass relationship based on inheritance. A general class (called the supertype) is modeled and then divided into several specialized classes (called subtypes). Inheritance is called “is-a” relationship.



Inheritance

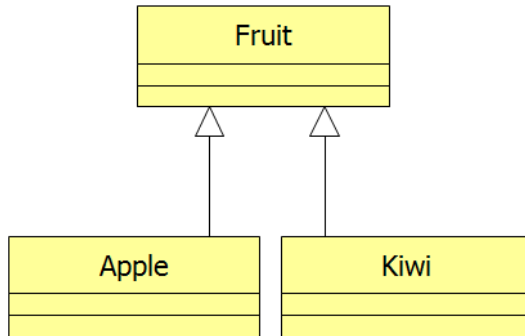


Figure 7.4: Inheritance Example



Inheritance

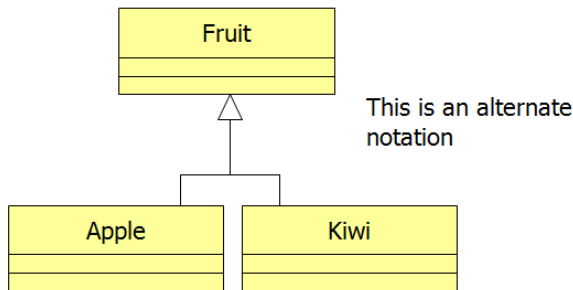


Figure 7.5: Another Inheritance Example



Multiple Inheritance

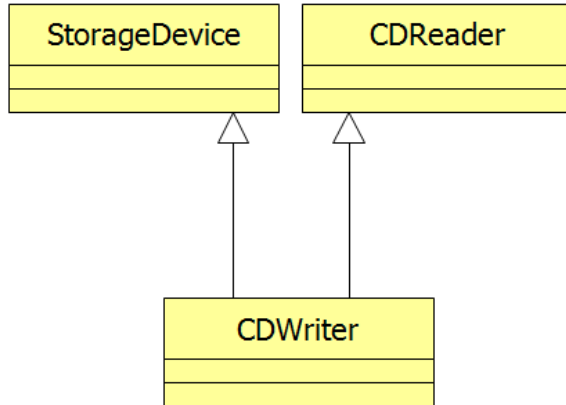


Figure 7.6: Multiple Inheritance



7.3.2. Groups

Whenever we create a subclass, we are defining a new group of objects that is a subset of the superclass' group. To be even more complete, when we define inheritance, we usually use keywords to give details about those groups; Overlapping/Disjoint

- Overlapping:
 - Groups are overlapping if there are objects that are members of both groups.
- Disjoint:
 - Groups are disjoint if there are no objects that are members of both groups.
- Example: Say we have a superclass called 'Movie'. Say we have two subclasses of 'Movie' called 'Comedy' and



‘Action’. Since it is possible for a movie to have both action and comedy, the two groups are overlapping.

Overlapping Groups

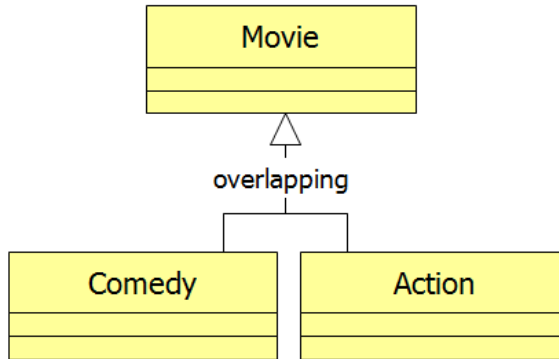


Figure 7.7: Overlapping Groups



BIT 2114 Object Oriented Analysis and Design

Overlapping/Disjoint: Example: Say we have a superclass called 'Book'. Say we have two subclasses of 'Book' called 'Paperback' and 'Hardcover'. Since a book is either a paperback or a hardcover book, the two groups are disjoint.



Disjoint Groups

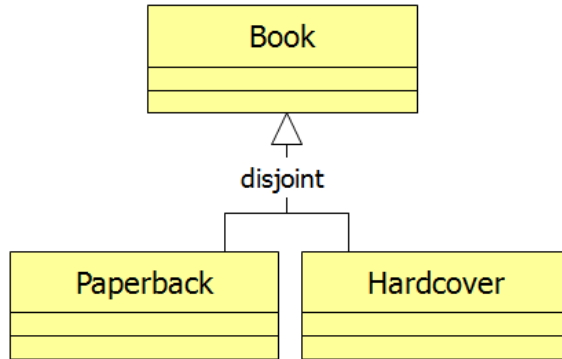


Figure 7.8: Disjoint Groups

Complete/Incomplete

- Complete: The groups listed contain all possible objects



BIT 2114 Object Oriented Analysis and Design

that belong to the superclass' group i.e. No more disjoint subclasses can be defined.

- Incomplete: The groups listed do not contain all possible objects that belong to the superclass' group.
- Example: Consider our superclass 'Book' with its subclasses 'Hardcover' and 'Paperback'.
- Are there any other kinds of book?
 - The answer is 'no', this the groups are complete.



Complete Groups

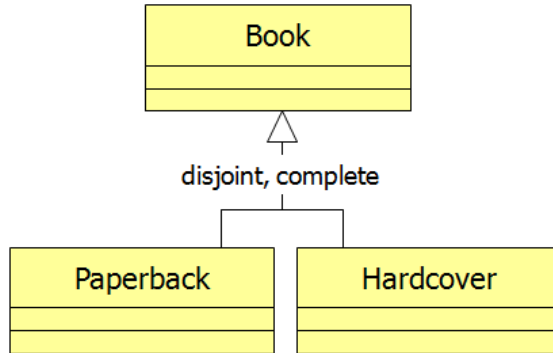


Figure 7.9: Complete Groups



BIT 2114 Object Oriented Analysis and Design

Complete/Incomplete: Example: Consider our superclass 'Movie' with its subclasses 'Comedy' and 'Action'.

- Are there any other kinds of movie?
- The answer is 'yes', this the groups are incomplete.



Incomplete Groups

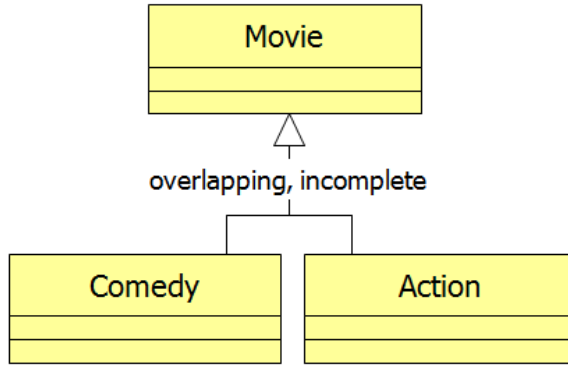


Figure 7.10: Incomplete Groups



7.3.3. Class Diagram: Associations

Relationship where no object is subordinate to others. It is a link between instances (objects) of classes. Associations between classes represent logical relationships objects of those classes might hold.

For example, a 'Person' class might have an association 'Home-Address' with another class 'Location'. This is because an instance of Person might have an instance of Location as its home address. Keep in mind, associates deal with objects, rather than classes. Thus they declare what associations are possible between their instances. There may be several instances of the association at any time. The number of instances changes over time. One instance may hold several instances of the same association.



BIT 2114 Object Oriented Analysis and Design

An association contains:

- The name of the association
 - This name describes the association
- Multiplicity at each end of the association
 - This describes the number of instances that may occur at that end of the association
- The role of the class at each end (optional)
 - This is usually a name used to describe the instances with respect to that association

Let's illustrate these concepts with an example: Say we have two classes, 'Person' and 'MusicalInstrument', and an association called 'Plays'. The association Plays represents all instances where a Person plays a Musical Instrument.



Association



Figure 7.10: Association Example

- We might have the following instances of ‘Plays’:
 - Bob plays Piano.
 - Lucy plays Saxophone.
 - Lucy plays Trumpet.
 - Sue plays Drums.
- The multiplicity of Person means:
 - How many people can play a given instrument? What is the answer?



BIT 2114 Object Oriented Analysis and Design

- Any number of people can play a given instrument, including zero. This is represented using 0..*
- The multiplicity of MusicalInstrument means: How many instruments can a person play?
 - What is the answer? A person may play zero instruments, or any number of them. Again, this is 0..*
- Essentially, an association could be considered a class itself. Every instrument any person plays is an instance of the Plays association.
- Associations would have members such as 'name', 'multiplicity1', 'multiplicity2', etc..

Association

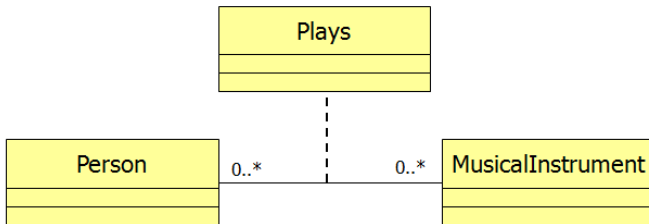


Figure 7.11: Another Association Example

Associations need not always be between 2 classes: Associations can exist between 3 or more classes as well.



7.3.4. Class Diagram: Composition

Composition is a relationship between two classes where one class is a part of another (an 'is a part of' relationship): If a class A is composed of classes B, C, and D, we may say that A has components B, C, and D. Compositions typically consist of a name, as well as the multiplicity of the component. If a class 'Car' has a composition relationship with another class 'Wheel', we should agree that normally the multiplicity of Wheel is 4 (exactly). 'Vehicle', on the other hand, could have any number of wheels (0..*). Composition is a special form of association. It is important enough to have its own notation.



Composition

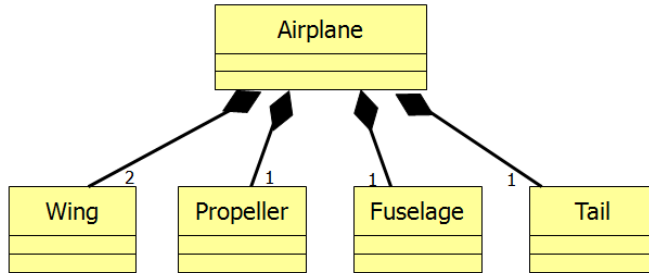


Figure 7.12: Composition Example

The black diamond represents composition. It is placed on the Airplane class because it is the Airplane that is composed of a Wing, Propeller, Fuselage and Tail.



7.3.5. Class Diagram: Aggregation

An aggregation is also a type of association, but again, a special one that is given its own notation. This is because, like composition, it is very common. An association that models a whole-part relationship between an aggregate, which is called whole, and its parts. It is a strong form of association in which the aggregate has no meaning without parts, hence aggregation is known as “part-of” relationship e.g computer made up of parts. An aggregation is another relationship between classes which says one class ‘is a part of’ another class.

Both aggregation and composition represent the ‘is a part of’ relationship. The main difference is what the part represents. In aggregation, the part (constituent) is meaningful without the whole (aggregate). In composition, the part (component) is not



BIT 2114 Object Oriented Analysis and Design

meaningful without the whole (container). A side-effect of this difference is that a component is always a part of (at most) one container. However, a constituent may be a member of multiple aggregates.

In UML, composition is considered a special case of aggregation where constituents belong only to a single aggregate. This is why some tools (such as Rational Rose) only have symbols for aggregation, and not composition.

To illustrate a constituent, again, let's use a few examples:

- A Forest is an aggregate of Trees. o
- A Program is an aggregate of Statements.
- A Fleet is an aggregate of Ships.
- A Deck is an aggregate of Cards.



BIT 2114 Object Oriented Analysis and Design

Parts of an aggregate are called constituents:

- A Tree is an constituent of Forest.
- A Statement is an constituent of Program.
- A Ship is an constituent of Fleet.
- A Card is an constituent of Deck.

These constituents share one common property: The constituents of an aggregate are usually the same type. Two other properties are also defined for aggregation:

- An object may be a constituent of more than one aggregate simultaneously. It is possible to have an aggregate without any constituents
- An object may be a constituent of more than one aggregate simultaneously. This is identified by the multiplicity of the



BIT 2114 Object Oriented Analysis and Design

aggregate end of the association.

It is possible to have an aggregate without any constituents. This is identified by the multiplicity of the constituent end of the association.

The weak form of aggregation is denoted with an open diamond. This relationship denotes that the aggregate class (the class with the white diamond touching it) is in some way the “whole”, and the other class in the relationship is somehow “part” of that whole.

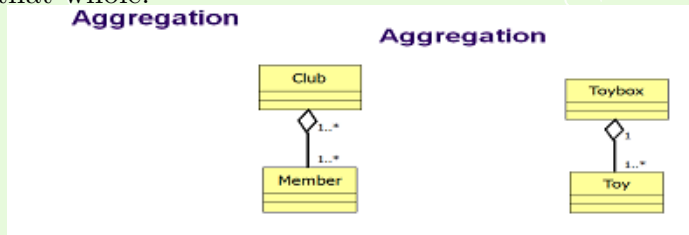


Figure 7.13: Aggregation Example

Aggregates can be ordered or unordered. In ordered aggregates, the order of the constituents within the aggregate is important. In unordered aggregates, the order is unimportant. Ordered aggregates are indicated using the symbol [ordered], while unordered is the default for aggregates.





Aggregate Order

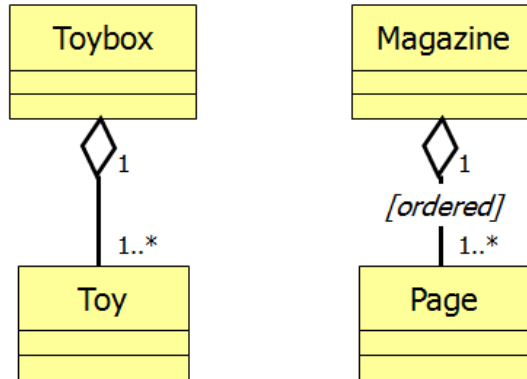


Figure 7.14: Aggregate Order

In programming, aggregation and composition are usually



BIT 2114 Object Oriented Analysis and Design

represented by class attributes. However, for clarity, in class diagrams, aggregation and composition should always be used instead of the attribute form. This is because it allows the definition of the class being used to be shown. Say we are writing a word processor: A word processing document consists of a number of paragraphs. Each paragraph consists of a number of elements. An element can be a character or an image.



Class Diagrams: An Example

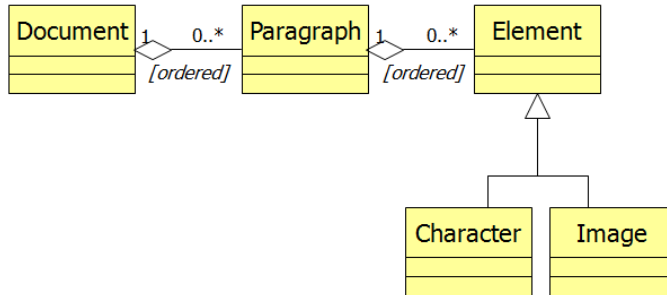


Figure 7.15: Class Diagram

7.4. Example

Every student at JKUAT can be a member of none or many clubs. Each club must have at least one student as a member,



BIT 2114 Object Oriented Analysis and Design

besides the club officials. Each club must have at least three club officials and just one chairperson. Each club official can be an official to any other club(s). Draw a class diagram showing at least one attribute for each class.

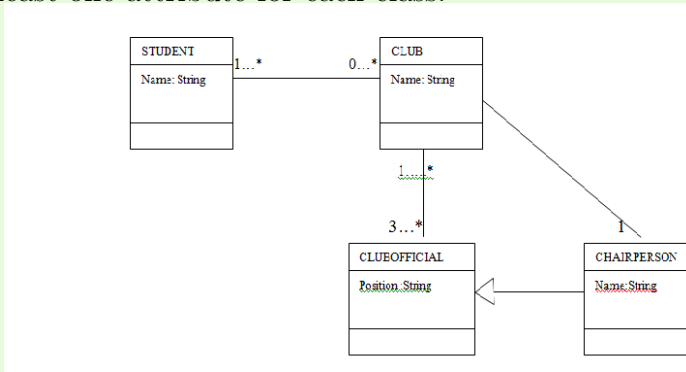


Figure 7.15: Club Class Diagram



7.5. Class Stereotypes

Stereotypes are the core extension mechanism of the UML. If you find that you need a modeling construct that isn't in the UML but is similar to something that is, you treat your construct as a stereotype of the UML construct. A stereotype name is surrounded by Gullimments symbols << >>

An analysis class can be classified into one of the following there stereotypes:

1. Entity classes: Used to model information and associated behavior of some phenomenon or concept such as an individual, a real-life object, or a real-life event.
2. Boundary classes: Used to model interaction between the system and its actors.

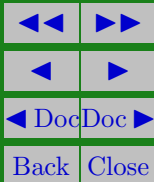


BIT 2114 Object Oriented Analysis and Design

3. Control classes: Used to represent coordination, sequencing, transactions and control of other objects.

Consider the class diagram for Add a New Advert to a campaign.

JKUAT SODEL
©2014





<<Boundary>>

User Interface: Add Advert
U1

Start interface ()

Assign staff ()

Select client ()

Select campaign ()



Figure 7.16: Class Stereotypes

7.5.1. Boundary

This is a stereotype class that provides an interface between users and other systems. It models interactions between the system and its actors.

Stereotypes are relatively abstract i.e. they do not directly represent all sorts of interfaces such as buttons, windows, menus, list boxes, e.t.c. that appears on the screen that will be used in the implementation language.

With these stereotypes, we are only interested with identifying the main logical interface with other software and also with physical devices such as monitors, sensors, printers, e.t.c. E.g. the class user interface: Add Advert U1 shown in the collabora-

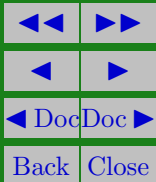




JKUAT SODEL
©2014

BIT 2114 Object Oriented Analysis and Design

tion and class diagram is a typical boundary class that can be illustrated in three ways as follows:





<<Boundary>>

User Interface: Add Advert
U1

Start interface ()

Assign staff ()

Select client ()

Select campaign ()



<<Boundary>>

User Interface: Add Advert

Start interface ()

Assign staff ()

Select client ()

Select campaign ()





Figure 7.17: Class Boundary

7.5.2. Entity Classes

This is a stereotype class that represents objects in the business domain model. They are used to model information and associated behavior of some phenomenon or concept, such as an individual, a real life object or a real life event as a general rule. Entity classes represent something within the application domain but external to software system about which the system must store information. E.g. an entity class campaign can be shown in three ways as follows:



<<Entity>>

Campaign

Title

Campaign startDate

Campaign FinishDate

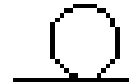
Get campaign Advert ()

Add New Advert ()



<<Entity>>

Campaign



Title

Campaign startDate

Campaign FinishDate

Get campaign Advert ()

Add New Advert ()



Figure 7.17: Entity Class

7.5.3. Control Classes

This is a stereotype class that controls the interaction between boundary classes and entity classes. It represents coordination sequence transactions and control of other objects. In USDP its generally recommended that there should be a control class for each use case. Control class represents a calculation and scheduling aspects of the logic of the use case. The control class can be shown in three ways as follows.



<<Control>>

Control: Add Advert

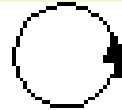
Show client campaign ()

Show client campaign Advert ()

Create New Advert ()



<<Control>>



Control Add Advert

Title

Campaign startDate

Campaign FinishDate

Get campaign Advert ()

Add New Advert ()



Figure 7.18: Control Class

7.6. Summary

In class diagrams, there are three basic types of classes:

1. Entity classes: Used to model information and associated behavior of some phenomenon or concept such as an individual, a real-life object, or a real-life event.
2. Boundary classes: Used to model interaction between the system and its actors.
3. Control classes: Used to represent coordination, sequencing, transactions and control of other objects.



Learning Activities

Use the following Description of the Classes to draw a Class Diagram for the class timetabling system.

The classes defined for the JKUAT class timetabling system are: User, Primary User, Timetabling Master, Maintainers (IT & Software developer), Login GUI, Security System, Database Manager (DB), Timetabling System, User Details DB, Timetabling Database, Printer and Disk.

The timetabling system shall be connected to two main databases: the database with user login details and the database with timetabling details such as courses information, room information and so on. The UserDetailsDB class connects to the User details database. The timetableDB class connects to the database with timetabling details.

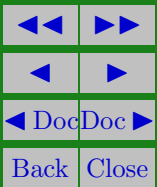


BIT 2114 Object Oriented Analysis and Design

The DBmanager class opens a connection to the database server, sends a query and closes the connection. The Timetabling system class gets users queries, and gets the result from the timetable database. This class also accepts data to be input and modified, which is subsequently updated to the database.

The User class represent a general user of the system, a user can have the role of primary user (student, instructor, other university staff), timetabling master or IT department. This class will store all common data and functions of users. The login function will call the Security class to validate the user's details. The logout functions will simply logout the user.

The Login GUI class represents the login GUI window which is opened by the user. Here, the user simply enters their login details and if log in details are wrong, gets the error message.





BIT 2114 Object Oriented Analysis and Design

The user can log out from this window.

The Security System class provides user validation which checks user details from the database. When user is validated, Userwindow is called to display the relevant window for the user depending on their security level, otherwise ErrorLoginMsg is called. Many users can be validated by one security system. Once a user is validated, the useWindow class displays the relevant window for users, depending on their security levels. A user can exit from this window.

The primary user class provides the user with possibility to query the timetable, print or save queried data.





Revision Questions

Example . What is the difference between an aggregation and an association?

Solution: for revision



EXERCISE 1.  Describe the FIVE steps involved in developing a class diagram from a problem description

EXERCISE 2.  With the help of a diagram explain the characteristics of a class

EXERCISE 3.  Draw a class diagram to represent a company that meets the following description: ABC corporation has several employees. For each employee the company keeps track of that employee's last name, first name, middle name, gen-



BIT 2114 Object Oriented Analysis and Design

der, home address, date hired and birth date. Some employees are salaried employees. For these employees, the company is interested in their annual salary. Other employees are hourly employees, the company is interested in knowing their hourly rate.



References and Additional Reading Materials

1. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403
2. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
3. Journal of computer science and Technology ISSN 1000-9000